

ICU init-ext Firmware Documentation

Generated by Doxygen 1.13.2

1 File Index	1
1.1 File List	1
2 File Documentation	3
2.1 src/invn/soniclib/sensor_fw/icu_init-ext/icu_init-ext.h File Reference	3
2.1.1 Detailed Description	4
2.1.2 Function Documentation	5
2.1.2.1 icu_init_ext_display_config_info()	5
2.1.2.2 icu_init_ext_get_iq_start_addr()	5
2.1.2.3 icu_init_ext_read_addr()	6
2.1.2.4 icu_init_ext_trim_rx_length()	7
2.1.2.5 icu_init_ext_update_metadata_from_iq0()	8
2.1.2.6 icu_init_ext_write_addr()	9
2.1.2.7 icu_init_ext_write_cfg_sequence()	9
2.1.2.8 icu_init_ext_write_seq()	10
2.2 icu_init-ext.h	11

Chapter 1

File Index

1.1 File List

Here is a list of all documented files with brief descriptions:

src/invn/soniclib/sensor_fw/icu_init-ext/ icu_init-ext.h	
This file defines the API for the init-ext firmware	3

Chapter 2

File Documentation

2.1 src/invn/soniclib/sensor_fw/icu_init-ext/icu_init-ext.h File Reference

This file defines the API for the init-ext firmware.

```
#include <invn/soniclib/details/icu.h>
#include <invn/soniclib/soniclib.h>
#include <stdint.h>
#include <invn/icu_interface/shasta_pmut_instruction.h>
#include <invn/icu_interface/shasta_external_regs.h>
#include <invn/soniclib/sensor_fw/icu_init-ext/icu_algo_format.h>
#include <invn/soniclib/sensor_fw/icu_init-ext/icu_shasta_algo_structs.h>
#include <invn/soniclib/sensor_fw/icu_init-ext/shasta_init_interface.h>
```

Macros

- **#define ICU_INIT_EXT_MAX_SAMPLES** (520)
- **#define ICU_INIT_EXT_MAX_INSTR_SEQ_LEN** (1000)
- **#define ICU_INIT_EXT_INSTR_SEQ_0_ADDR** (0x1008)
- **#define ICU_INIT_EXT_INSTR_SEQ_1_ADDR** (0x100A)
- **#define ICU_INIT_EXT_INSTR_SEQ_2_ADDR** (0x100C)
- **#define ICU_INIT_EXT_INSTR_SEQ_3_ADDR** (0x100E)
- **#define ICU_INIT_EXT_SEQ_TABLE_0_ADDR** (0x1010)
- **#define ICU_INIT_EXT_SEQ_TABLE_1_ADDR** (0x1012)
- **#define ICU_INIT_EXT_SEQ_TABLE_IDX_0_MASK** (0x0003)
- **#define ICU_INIT_EXT_SEQ_TABLE_IDX_1_MASK** (0x000C)
- **#define ICU_INIT_EXT_SEQ_TABLE_IDX_2_MASK** (0x0030)
- **#define ICU_INIT_EXT_SEQ_TABLE_IDX_3_MASK** (0x00C0)
- **#define ICU_INIT_EXT_SEQ_TABLE_IDX_4_MASK** (0x0300)
- **#define ICU_INIT_EXT_SEQ_TABLE_IDX_5_MASK** (0x0C00)
- **#define ICU_INIT_EXT_SEQ_TABLE_IDX_6_MASK** (0x3000)
- **#define ICU_INIT_EXT_SEQ_TABLE_IDX_7_MASK** (0xC000)

Functions

- uint16_t **get_icu_init_ext_fw_ram_init_addr** (void)
- uint16_t **get_icu_init_ext_fw_ram_init_size** (void)
- const unsigned char * **get_ram_icu_init_ext_init_ptr** (void)
- uint8_t **icu_init_ext_init** (ch_dev_t *dev_ptr, fw_info_t **fw_info)
- uint16_t **icu_init_ext_read_addr** (ch_dev_t *dev_ptr, uint8_t instr_seq_idx)

Read the address of the instruction sequence with index `instr_seq_idx`.
- void **icu_init_ext_write_addr** (ch_dev_t *dev_ptr, uint8_t instr_seq_idx, uint16_t addr)

Write the address of the instruction sequence extension with index `instr_seq_idx`.
- int **icu_init_ext_write_seq** (ch_dev_t *dev_ptr, uint16_t addr, const pmut_transceiver_inst_t *seq)

Write the instruction sequence pointed to by `seq` to address `addr`.
- int **icu_init_ext_trim_rx_length** (pmut_transceiver_inst_t *orig_instr_seq, uint8_t odr, pmut_transceiver_inst_t *instr_seq)

This function trims the total RX length to remove excess ADC samples that don't result in additional IQ sample.
- uint16_t **icu_init_ext_get_iq_start_addr** (const ch_dev_t *dev_ptr)

Get the address of the first element of the IQ data buffer in memory.
- uint8_t **icu_init_ext_display_config_info** (ch_dev_t *dev_ptr, pmut_transceiver_inst_t *instr_ext0, pmut_transceiver_inst_t *instr_ext1, pmut_transceiver_inst_t *instr_ext2, pmut_transceiver_inst_t *instr_ext3)

Print sensor configuration info.
- uint8_t **icu_init_ext_write_cfg_sequence** (ch_dev_t *dev_ptr, uint8_t *seq, uint8_t seq_len)

Write the order in which measurement configurations are executed.
- uint8_t **icu_init_ext_update_metadata_from_iq0** (ch_dev_t *dev_ptr, ch_iq_sample_t *iq_data, uint8_t *seq, uint8_t seq_len, uint8_t *cur_meas)

Extract the metadata from the first IQ sample and update the device pointer.

Variables

- const char * **icu_init_ext_version**
- const uint8_t **icu_init_ext_fw_text** []
- const uint8_t **icu_init_ext_fw_vec** []
- const uint16_t **icu_init_ext_text_size**
- const uint16_t **icu_init_ext_vec_size**

2.1.1 Detailed Description

This file defines the API for the init-ext firmware.

The init-ext firmware expands the normal init firmware with support for extended instruction sequences beyond the original 32. Additionally it supports a total of four measurement configurations. It was not possible to do this without allocating memory previously used for the IQdata to the instruction sequence extensions, which is the approach this firmware variant uses.

Note that transmit optimization is NOT supported with the init-ext firmware.

2.1.2 Function Documentation

2.1.2.1 icu_init_ext_display_config_info()

```
uint8_t icu_init_ext_display_config_info (
    ch_dev_t * dev_ptr,
    pmut_transceiver_inst_t * instr_ext0,
    pmut_transceiver_inst_t * instr_ext1,
    pmut_transceiver_inst_t * instr_ext2,
    pmut_transceiver_inst_t * instr_ext3)
```

Print sensor configuration info.

This function serves the same purpose as `ch_display_config_info()`, but it is designed to support the instruction extensions.

Parameters

in	<i>dev_ptr</i>	The device pointer.
in	<i>instr_ext0</i>	Pointer to the first instruction extension (or NULL)
in	<i>instr_ext1</i>	Pointer to the second instruciton extension (or NULL)
in	<i>instr_ext2</i>	Pointer to the third instruction extension (or NULL)
in	<i>instr_ext3</i>	Pointer to the fourth instruciton extension (or NULL)

Returns

Zero for success.

2.1.2.2 icu_init_ext_get_iq_start_addr()

```
uint16_t icu_init_ext_get_iq_start_addr (
    const ch_dev_t * dev_ptr)
```

Get the address of the first element of the IQ data buffer in memory.

This function can be used as a convenience to get the address of the IQ data buffer in memory. The instruction sequences are contiguous with the IQ data, and the user may change the allocation by modifying the addresses of the instruction extensions. This function is useful for finding the lower boundary of the usable space.

Parameters

in	<i>dev_ptr</i>	The device pointer.
----	----------------	---------------------

Returns

The address of the first IQ sample.

Note

This function does not perform any IO. It must be called after `ch_group_start()`. As of init-ext firmware version 1.1 (or plugin version 2.2), the IQ data will always be found at address 0x11e4.

2.1.2.3 `icu_init_ext_read_addr()`

```
uint16_t icu_init_ext_read_addr (
    ch_dev_t * dev_ptr,
    uint8_t instr_seq_idx)
```

Read the address of the instruction sequence with index `instr_seq_idx`.

This function retrieves the address of the start of the instruction sequence with index `instr_seq_idx`. There are two instruction sequence extensions available, one for each of the original 32-length sequences. Additionally, there are two independent instruction sequences available. The user may write instructions to the sequences by writing to the address returned by this function. See [icu_init_ext_write_addr\(\)](#).

We will refer to these added instruction sequences as extensions (or ext) 0-3. Note that ext0 and ext1 extend the original instruction sequences, while ext2 and ext3 are independent instruction sequences. This brings the total number of independent configurations to 4.

When using ext2 and ext3, you may no longer use independent ODR settings per config; these are hard coded to use the meas1 (ext1) ODR. You may no longer use free-running mode. You must also not use the data ready interrupt suppression feature.

By default, these addresses are configured the following way:

- The ext0 address is 4 bytes after the last IQ sample in the IQ data buffer. That is, the IQ length in bytes can be found by subtracting the IQ start address from the ext0 address. By default, the IQ data buffer length is $520 \times 4 = 2080$. You should not assume that the default ext0 length is fixed. It is subject to change when the init-ext firmware is modified.
- The ext1 address points to the last 4-byte word available for extension. That is, ext1 is configured for a default length of 1. The available total space for extensions (in bytes) is then $\text{ext_addr}(1) - \text{ext_addr}(0) + 4$.
- The address of ext2 and ext3, which are independent sequences not tied to the original configurations, is also set to the last 4-byte word available.

By configuring the defaults this way, it is possible to determine the available memory for combined IQdata + extensions as $\text{ext_addr}(1) - \text{iq_start_addr} + 4$. The IQ start address can be found with [icu_init_ext_get_iq_start_addr\(\)](#).

In summary, at start-up, the memory is organized the following way:

- 0x1000: Address of top level sensor configuration = $\&\text{IQdata}[0] - 0x1d0$
- 0x1008: Address of ext0 = $\&\text{IQdata}[0] + 2080$
- 0x100A: Address of ext1 = Address of last available word
- 0x100C: Address of ext2 = Address of last available word
- 0x100E: Address of ext3 = Address of last available word
- $\ast(0x1000) + 0x1d0$: Start of IQdata
- $\ast(0x1000) + 0x1d0 + 2080 = \ast(0x1008)$: Instruction sequence extension 0
- $\ast(0x100A)$: Instruction sequence extension 1 (default length 1)
- $\ast(0x100C)$: Instruction sequence extension 2 (default length 1)
- $\ast(0x100E)$: Instruction sequence extension 3 (default length 1)

In double buffer mode, the address of the second IQ buffer will be adjusted to mid-point of the IQ start address and the extension 0 address. That is, $\text{second_iq_buf_addr} = (\text{ext0_addr} + \text{iq_start_addr}) / 2$.

Parameters

in, out	<i>dev_ptr</i>	The pointer to the device.
in	<i>instr_seq_idx</i>	The index of the sequence. Valid values are 0, 1, 2, and 3.

Returns

The address of the instruction sequence extension.

Note

As of init-ext firmware 1.1 (plugin version 2.2), the IQ data location is fixed in memory and can be found at address 0x11e4.

2.1.2.4 icu_init_ext_trim_rx_length()

```
int icu_init_ext_trim_rx_length (
    pmut_transceiver_inst_t * orig_instr_seq,
    uint8_t odr,
    pmut_transceiver_inst_t * instr_seq)
```

This function trims the total RX length to remove excess ADC samples that don't result in additional IQ sample.

The ICU-x0201 parts use an oversampling ADC. The PMUT state machine time-base is in terms of the ADC clock. That is, when you set the length of the instruction, you are specifying the length in ADC clock cycles. There are many values of total RX length that result in the same number of total IQ samples. This function trims the excess RX length.

It is required to run this function before loading the config to avoid particular values of RX length that can cause some undesirable or unexpected behavior.

This function requires both the original instruction sequence, the ODR, and the extension. It gets the total RX length from analyzing both of these, then applies the trim to the extension. If no extension is provided or if the extension is not accessible, the trim is applied to the original instruction sequence.

This function returns the resulting number of IQ samples post-trimming. If called more than once on the same input, it will not perform additional trimming and can be used to simply get the number of samples.

Parameters

in	<i>orig_instr_seq</i>	For ext0 and ext1: the original instruction sequence the extension applies to. For ext2 and ext3: NULL.
in	<i>odr</i>	The measurement ODR. For ext2 and ext3: must match ext1 ODR.
in, out	<i>instr_seq</i>	The extension that will have the RX length trimmed.

Returns

The number of IQ samples, post-trimming.

Return values

-1	The provided configuration is invalid (missing either EOF or RDY_IEN).
-2	The function <code>chdrv_adjust_rx_len()</code> returned an error. See that function's documentation.
-3	The provided <code>instr_seq</code> is NULL, but no EOF or RDY_IEN was found in <code>meas_config</code> .

Note

This step is normally performed by `chdrv_meas_queue_write()`, but we want to bypass the default implementation since that also affects the IEN bits in the instructions.

2.1.2.5 `icu_init_ext_update_metadata_from_iq0()`

```
uint8_t icu_init_ext_update_metadata_from_iq0 (
    ch_dev_t * dev_ptr,
    ch_iq_sample_t * iq_data,
    uint8_t * seq,
    uint8_t seq_len,
    uint8_t * cur_meas)
```

Extract the metadata from the first IQ sample and update the device pointer.

The ICU-X0201 can be configured to place metadata in the first IQ sample, which is otherwise always read out as 0. If enabled, this will place the IQ buffer address as well as the last measurement index in the first IQ sample. See `ch_enable_metadata_in_iq0()`.

This function is used to extract metadata from the read IQ data. The metadata will be updated into `dev_ptr`. That is, after calling this function, the device pointer will be updated with the correct buffer address and last measurement index.

After running this function and getting a 0 exit status, the last measurement index can be retrieved using `ch_meas_get_last_num()` and the next buffer address with `ch_get_next_buf_addr()`.

This function fulfills the same purpose as `ch_update_metadata_from_iq0()`, but the implementation is different due to how the `icu-init-ext` firmware handles the IQ metadata vs the GPT and normal init firmware. The `icu-init-ext` firmware writes the value of `cur_meas` to the metadata, whereas the others write the index of the measurement configuration actually used.

When calling this function, `dev_ptr->last_measurement` is still updated with the index of the measurement configuration. The raw value of `cur_meas` is obtained by passing in a pointer for the `cur_meas` argument. This is useful for checking if an application reading data from the sensor ever gets out of sync with the expected value of `cur_meas`.

Parameters

<i>dev_ptr</i>	The device pointer
<i>iq_data</i>	Pointer to the read IQ data. The metadata is extracted from the first sample, then the first sample is set back to 0.
<i>seq</i>	The previously programmed sequence of configs. This is needed to map the <code>cur_meas</code> value from the metadata to a meas config index. If NULL is passed and <code>seq_len</code> is 0, the default implementation of config0 for <code>cur_meas=0</code> and config1 otherwise is assumed.
<i>seq_len</i>	The length of <code>seq</code> . Pass 0 if <code>seq</code> is NULL.
<i>cur_meas</i>	The extracted value of <code>cur_meas</code> .

Returns

0 for success and non-zero otherwise. If this function returns non-zero, then the device pointer is not updated. This most likely means that data was read from the incorrect buffer in double buffer mode. The situation should resolve itself on the next read since the same buffer will be read again, and the ASIC will swap write buffers on each measurement.

2.1.2.6 icu_init_ext_write_addr()

```
void icu_init_ext_write_addr (
    ch_dev_t * dev_ptr,
    uint8_t instr_seq_idx,
    uint16_t addr)
```

Write the address of the instruction sequence extension with index `instr_seq_idx`.

This function sets the address of the start of the instruction sequence extension with index `instr_seq_idx`. There are two instruction sequence extensions available, one for each of the original 32-length sequences. Additionally, there are two independent instruction sequences available. The user may write instructions to the extensions by writing to the address set by this function.

The addresses written need to fall within the available memory range, which can be found by examining the default values of the addresses. See [icu_init_ext_read_addr\(\)](#).

Parameters

in, out	<i>dev_ptr</i>	The pointer to the device.
in	<i>instr_seq_idx</i>	The index of the sequence. Valid values are 0, 1, 2, and 3.
in	<i>addr</i>	The address to write.

2.1.2.7 icu_init_ext_write_cfg_sequence()

```
uint8_t icu_init_ext_write_cfg_sequence (
    ch_dev_t * dev_ptr,
    uint8_t * seq,
    uint8_t seq_len)
```

Write the order in which measurement configurations are executed.

The icu-init-ext firmware supports programing a custom order of measurement configurations. This means that the ASIC itself can autonomously run the (up to) four measurement configurations in the desired order.

Note: Here we use the term "measurement configuration", but strictly only the first two available configurations are full "measurement configurations". The third and fourth configurations consist only of the instruction sequence portion of a full configuration. They use the ODR setting of the second full measurement configuration (`meas1`). When used, the free-running mode and data suppression interrupt feature are not supported.

The ASIC has an internal count variable called `cur_meas` which is incremented after each measurement is performed. It is initialized to the value `meas_start` and increments to the value `meas_stop`. Upon the next increment, it resets to `meas_start`.

By default, measurement configuration 0 is run when `cur_meas`=0 and measurement configuration 1 is run otherwise. Using this function, this behavior can be changed. The sequence programmed here maps the `cur_meas` value to a desired configuration index.

For example, if the sequence {2, 1, 3} is programmed, configuration 2 (ext2) is run first, followed by config 1, followed by config 3 (ext3). Then the pattern would repeat.

Parameters

in	<i>dev_ptr</i>	The device pointer.
in	<i>seq</i>	The desired order of measurement configuration execution
in	<i>seq_len</i>	The length of the sequence. Must be 16 or less.

Returns

An error code.

Return values

0	Success.
1	Passed sequence length is too long. Max length is 16.

2.1.2.8 icu_init_ext_write_seq()

```
int icu_init_ext_write_seq (
    ch_dev_t * dev_ptr,
    uint16_t addr,
    const pmut_transceiver_inst_t * seq)
```

Write the instruction sequence pointed to by seq to address addr.

This function writes the instruction sequence pointed to by seq to the address addr. The sequence MUST end with the EOF instruction. This is required to set the length of the written instruction sequence.

For ext2 and ext3, the situation is simple. Whatever is written to the new sequences will be executed when those instruction sequences are selected. For ext0 and ext1, the situation is more complex because these extend original sequences. The remainder of the documentation below applies to ext0 and ext1 only.

To enable the extension, the last instruction to be executed from the original configuration MUST have the RDY_IEN bit set. This causes the init-ext firmware to switch to the instruction extensions. No instructions after the one with RDY_IEN set will be executed; flow will continue to the first extension instruction.

The instruction with RDY_IEN set MUST be at least 10 clock cycles long. This is to give the software on the MCU time to switch the instruction pointer to the extension address after receiving the RDY type interrupt.

RDY_IEN is bit 2 of the PMUT command. The following example shows how to set this.

```
measurement_queue_t my_queue = { ... };
my_queue.meas[0].trx_inst[i].cmd_config |= (1 << 2);
```

Additionally, the original configuration MUST still contain the EOF instruction. This means that up to 31 of the original 32 instructions will be usable.

The following order is recommended to successfully enable the extension(s):

1. Initialize the sensor up to immediately before the call to `ch_meas_import()`.
2. Use `icu_init_ext_trim_rx_len()` to sanitize the RX length, providing the original measurement config, as well as the extension for all configurations you plan to use.
3. Configure the extension addresses using `icu_init_ext_<read|write>_addr()`
4. Write the extension(s) using `icu_init_ext_write_seq()`.
5. Continue with `ch_meas_import()`. Again, make sure the config specified with this call has the RDY_IEN bit set in the last instruction to be executed. Additionally, an EOF instruction is required in this configuration, even though it's not executed.

Parameters

in, out	<i>dev_ptr</i>	The pointer to the device.
in	<i>addr</i>	The start address.
in	<i>seq</i>	The sequence to write.

Returns

An error code.

Return values

0	No error.
1	Write not successful. No EOF instruction found in the extension.

Warning

There is no validation that the range of written addresses falls into the allowable range, so the user must be careful to only write to valid addresses. See `icu_init_ext_read_addr()` for a description of the memory layout.

2.2 icu_init-ext.h

[Go to the documentation of this file.](#)

```

00001
00014
00015 /*
00016 <!-- BEGIN INVN LICENSE -->
00017 TDK InvenSense 5 Clause License
00018
00019 Copyright (c) 2025, Invensense, Inc.
00020
00021 All rights reserved.
00022
00023 Redistribution and use in source and binary forms, with or without modification,
00024 are permitted provided that the following conditions are met:
00025
00026 1. Redistributions of source code must retain the above copyright notice, this
00027 list of conditions and the following disclaimer.
00028
00029 2. Redistributions in binary form must reproduce the above copyright notice,
00030 this list of conditions and the following disclaimer in the documentation and/or
00031 other materials provided with the distribution.
00032
00033 3. Neither the name of the copyright holder nor the names of its contributors
00034 may be used to endorse or promote products derived from this software without
00035 specific prior written permission.
00036
00037 4. This software, with or without modification, must only be used with a
00038 TDK InvenSense sensor.
00039
00040 5. Any software provided in binary form under this license, whether embedded
00041 in source code or provided as a compiled library or application, must not
00042 be reverse engineered, decompiled, modified and/or disassembled.
00043
00044 THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND
00045 ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
00046 WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE
00047 DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR
00048 ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
00049 (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;
00050 LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON
00051 ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
00052 (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS
00053 SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
00054 <!-- END INVN LICENSE -->
00055 */
00056
00057 #ifndef ICU_INIT_EXT_H_
00058 #define ICU_INIT_EXT_H_
00059
00060 #ifdef __cplusplus
00061 extern "C" {
00062 #endif
00063
00064 #include <invn/soniclib/details/icu.h>
00065 #include <invn/soniclib/soniclib.h>
00066 #include <stdint.h>
00067 #include <invn/icu_interface/shasta_pmut_instruction.h>
00068 #include <invn/icu_interface/shasta_external_regs.h>
00069 /* no algo selected - build minimal <invn/soniclib/sensor_fw/Init> config */
00070 #include <invn/soniclib/sensor_fw/icu_init-ext/icu_algo_format.h>
00071 #include <invn/soniclib/sensor_fw/icu_init-ext/icu_shasta_algo_structs.h>
00072 #include <invn/soniclib/sensor_fw/icu_init-ext/shasta_init_interface.h>

```

```

00073
00074 #define ICU_INIT_EXT_MAX_SAMPLES          (520)
00075 #define ICU_INIT_EXT_MAX_INSTR_SEQ_LEN    (1000) // some big number, used for searching for EOF
00076 #define ICU_INIT_EXT_INSTR_SEQ_0_ADDR     (0x1008)
00077 #define ICU_INIT_EXT_INSTR_SEQ_1_ADDR     (0x100A)
00078 #define ICU_INIT_EXT_INSTR_SEQ_2_ADDR     (0x100C)
00079 #define ICU_INIT_EXT_INSTR_SEQ_3_ADDR     (0x100E)
00080 #define ICU_INIT_EXT_SEQ_TABLE_0_ADDR     (0x1010)
00081 #define ICU_INIT_EXT_SEQ_TABLE_1_ADDR     (0x1012)
00082
00083 #define ICU_INIT_EXT_SEQ_TABLE_IDX_0_MASK (0x0003)
00084 #define ICU_INIT_EXT_SEQ_TABLE_IDX_1_MASK (0x000C)
00085 #define ICU_INIT_EXT_SEQ_TABLE_IDX_2_MASK (0x0030)
00086 #define ICU_INIT_EXT_SEQ_TABLE_IDX_3_MASK (0x00C0)
00087 #define ICU_INIT_EXT_SEQ_TABLE_IDX_4_MASK (0x0300)
00088 #define ICU_INIT_EXT_SEQ_TABLE_IDX_5_MASK (0x0C00)
00089 #define ICU_INIT_EXT_SEQ_TABLE_IDX_6_MASK (0x3000)
00090 #define ICU_INIT_EXT_SEQ_TABLE_IDX_7_MASK (0xC000)
00091
00092 extern const char *icu_init_ext_version; // version string in fw .c file
00093 extern const uint8_t icu_init_ext_fw_text[];
00094 extern const uint8_t icu_init_ext_fw_vec[];
00095 extern const uint16_t icu_init_ext_text_size;
00096 extern const uint16_t icu_init_ext_vec_size;
00097
00098 uint16_t get_icu_init_ext_fw_ram_init_addr(void);
00099 uint16_t get_icu_init_ext_fw_ram_init_size(void);
00100
00101 const unsigned char *get_ram_icu_init_ext_init_ptr(void);
00102
00103 uint8_t icu_init_ext_init(ch_dev_t *dev_ptr, fw_info_t **fw_info);
00104
00175 uint16_t icu_init_ext_read_addr(ch_dev_t *dev_ptr, uint8_t instr_seq_idx);
00176
00196 void icu_init_ext_write_addr(ch_dev_t *dev_ptr, uint8_t instr_seq_idx, uint16_t addr);
00197
00258 int icu_init_ext_write_seq(ch_dev_t *dev_ptr, uint16_t addr, const pmut_transceiver_inst_t *seq);
00259
00296 int icu_init_ext_trim_rx_length(pmut_transceiver_inst_t *orig_instr_seq, uint8_t odr,
00297                                pmut_transceiver_inst_t *instr_seq);
00298
00316 uint16_t icu_init_ext_get_iq_start_addr(const ch_dev_t *dev_ptr);
00317
00332 uint8_t icu_init_ext_display_config_info(ch_dev_t *dev_ptr, pmut_transceiver_inst_t *instr_ext0,
00333                                           pmut_transceiver_inst_t *instr_ext1, pmut_transceiver_inst_t
00334                                           *instr_ext2,
00335                                           pmut_transceiver_inst_t *instr_ext3);
00336
00369 uint8_t icu_init_ext_write_cfg_sequence(ch_dev_t *dev_ptr, uint8_t *seq, uint8_t seq_len);
00370
00415 uint8_t icu_init_ext_update_metadata_from_iq0(ch_dev_t *dev_ptr, ch_iq_sample_t *iq_data, uint8_t
00416                                                *seq, uint8_t seq_len,
00417                                                uint8_t *cur_meas);
00418
00418 #ifdef __cplusplus
00419 }
00420 #endif
00421
00422 #endif /* ICU_INIT_EXT_H_ */

```